

ICT PROJECT REPORT: Tumble-POP

Course: Introduction to Information & Communication Technologies (IICT) **Project:** 2D Game Development (C++ & SFML)

Students: Affan Khan (25I-0851) Ayan Waqas (25I-0901)

Date: December 7, 2025

1. Project Overview

Tumble-POP is a 2D arcade-style action platformer developed using C++ and the SFML library. The game puts the player in the shoes of a "Ghostbuster" character equipped with a vacuum device. The primary objective is to clear stages by sucking up various monsters Skeletons, Ghosts, and projectile-firing enemies and ejecting them, all while navigating complex environmental hazards.

This project was built to demonstrate proficiency in programming fundamentals, specifically focusing on logic building, memory management with dynamic arrays, collision handling, and state management without relying on advanced Object-Oriented Programming (OOP) features like Classes, strictly adhering to procedural programming paradigms.

2. Technologies and Tools Used

To create a visually engaging and functional game, the following tools were utilized:

- **Programming Language:** C++ (Standard 11/17).
- **Graphics Library:** SFML (Simple and Fast Multimedia Library) for rendering graphics, window management, and input handling.
- **Integrated Development Environment (IDE):** Visual Studio / VS Code.
- **Asset Management:**
 - **Leshy SpriteSheet Tool:** Used to map sprite sheets and calculate the exact **IntRect** coordinates (x, y, width, height) required for character and enemy animations.
 - **Banana Pro:** Used for generating high-quality 2D aesthetic assets and background art.

3. System Architecture & Data Structure

Since the project uses **Programming Fundamentals (PF)** concepts, complex Classes were avoided. Instead, the game relies on **Parallel Arrays** to manage game entities.

3.1. Entity Management

Instead of a single object for an enemy, data is split across multiple arrays sharing the same index `[i]`. For example, the **Invisible Man** enemy uses:

```
const int NUM_INVISIBLE = 3;
float inv_x[NUM_INVISIBLE];      // X Coordinate
float inv_y[NUM_INVISIBLE];      // Y Coordinate
float inv_speed[NUM_INVISIBLE];  // Movement speed
int inv_capture_state[NUM_INVISIBLE]; // 0=Walking, 1=Being Sucked, 2=Dead
bool inv_is_visible[NUM_INVISIBLE]; // Visibility flag
```

This approach allows for efficient iteration through loops to update physics and rendering for multiple entities simultaneously.

3.2. Game State Machine

The game flow is controlled by a simple integer state machine inside the main loop:

- **State 0:** Start Screen
- **State 1:** Character Select Menu (P1 / P2 selection)
- **State 3:** Level 1 (Standard Platforming)
- **State 4:** Level 2 (Advanced Physics/Slopes)
- **State 5:** Game Over

4. Technical Implementation & Custom Functions

The core gameplay logic is encapsulated in modular functions to maintain code readability. Below are the key functions developed for this project:

A. Physics and Movement

- **player_gravity(...)**: This function applies constant downward acceleration to the player. It uses predictive collision detection to check if the player's "feet" will hit a floor block (#) or a solid wall (@) in the next frame. If a collision is detected, it snaps the player to the top of the block and resets vertical velocity.
- **movement(...)**: Handles user input (Left, Right, Up, Down). It manages texture flipping (facing direction) and sprite animation frames. Crucially, it manages the **Texture Swapping** logic when the Spacebar is held, changing the player from "Walking" mode to "Vacuuming" mode.

- **handle_slope(...)**: A specialized function created for Level 2. It detects if the player is standing on a "Slanted Block" (represented by &). If detected, it modifies the player's X and Y coordinates simultaneously, creating a smooth sliding effect without complex vector mathematics.

B. Enemy Artificial Intelligence (AI)

- **spawn_enemy(...)**: A universal spawner that ensures enemies do not spawn inside walls or in mid-air. It checks the grid coordinates to ensure a valid floor exists beneath the spawn point.
- **update_enemy_logic(...)**: Controls enemy patrolling. It checks ahead of the enemy's current path. If a wall or a ledge (gap) is detected, the speed is multiplied by -1 , causing the enemy to reverse direction.
- **initialize_enemy_group(...)**: A helper function designed to initialize entire groups of enemies (Skeletons, Ghosts, Chelnovs) at once, assigning them textures and random starting directions.

C. Combat Mechanics

- **vacuumRangeCheck(...)**: This calculates the distance between the player and every active enemy. If an enemy falls within a specific pixel range (X and Y axis) while the vacuum is active, their state changes from 0 (Walking) to 1 (Being Sucked).
- **adjustVacuumSprites(...)**: Since the vacuuming sprite is larger than the walking sprite, switching textures caused the player to visually "jump" or shift. This function calculates an offset based on the facing direction to ensure the transition looks smooth and centered.
- **handle_chelnov_enemies(...)**: Manages the unique behavior of the "Chelnov" enemy, which includes a timer. Every 3 seconds (180 frames), it spawns a projectile bullet in the direction the enemy is facing.

D. Game Systems

- **generate_level2(...)**: Uses loops to procedurally generate specific patterns in the level array. It creates a hardcoded "Staircase" structure using coordinate mapping and fills the rest of the map with semi-random platforms, ensuring they don't overlap with the stairs.
- **spawn_powerups(...)**: Checks if player health is critical (≤ 2). If true, it spawns a random powerup (Speed, Health, or Range) at a valid location on the map.

5. Development Workflow

1. **Map Design:** The levels were designed using a 2D char array (`char** lvl`), where symbols represent game objects (`#` for bricks, `@` for walls, `&` for slopes).
2. **Asset Integration:** Sprites were cut using the **Leshy Tool** to get precise pixel coordinates for `IntRect`.
3. **Physics Implementation:** Gravity and collision were implemented first to ensure the character could traverse the map.
4. **Enemy Logic:** Basic patrolling AI was added, followed by the specific vacuum capture mechanic.
5. **Polish:** Level 2's unique slope mechanic and specific enemy types (Invisible Man, Chelnov) were added in the final phase.

6. Challenges and Solutions

- **Challenge:** The player would sometimes get stuck inside walls during high-speed movement.
 - **Solution:** We implemented a predictive collision check in `movement()` that looks ahead by `speed` pixels before actually moving the player.
- **Challenge:** The "Invisible Man" enemy needed to blink without disappearing for everyone simultaneously.
 - **Solution:** In the initialization loop, we assigned a random starting value to `inv_vis_timer[i]`, ensuring their invisibility cycles were desynchronized.

7. Future Enhancements

- **Sound Effects:** Integrating `sf::SoundBuffer` for vacuum and jump sound effects.
- **Boss Fight:** Adding a large boss enemy with a health bar for Level 3.
- **High Score Saving:** Using file handling (`fstream`) to save player scores permanently